

# gnss-scope 実装設計（202607）

実装の設計書（構成・データモデル・実装上の注意）。役割分担は次のとおり。

| 文書                                 | 内容                                  |
|------------------------------------|-------------------------------------|
| <code>funcspec-202607.md</code>    | 機能仕様（何をするか・仕様値）                     |
| <code>usersGuide-202607.md</code>  | 利用者の手引（操作手順・画面の読み方）                 |
| <code>design-202607.md</code> （本書） | 実装設計（画面実装・データモデル・モジュール構成・リファクタ記録）   |
| <code>algospec-202607.md</code>    | 算出式・自動判定ロジックの詳細                     |
| <code>hardware-202607.md</code>    | 機器・ファーム（Pico W / MAX-M10S / BLE）の仕様 |

本書は、前身 `gnss-tracker`（地理院地図の全画面表示がメイン）を `gnss-scope` として再構築した際の設計を出発点としている。受信パイプライン（BLE → 行復元 → NMEA パース → エポック確定 → 受信品質統計）の考え方は前身から引き継いでおり、**変更点は UI 構成・記録モデル・解析の対象**である。

## 1. 方針

- 地図をメインにしない。地図は5タブのうちの1つ。役割も「ライブ地図」から「load した記録の GNSS 値をマーカー表示する」へ移す（ライブ現在地表示は副機能として残す）。
- 記録は静止点のばらつき評価に一本化。1回の記録 = 1地点。旧「スナップショット記録（1点だけ即保存）」は廃止し、`record` → `stop` → `save` に統一。旧「静的測位記録」の収集・集計・収束自動停止ロジックはそのまま流用する。
- 解析は `SkyPlot` / `SNR` / `DOP` / `DRMS` の4本立てとし、ライブと読込データの双方を同じ画面で見る。

## 2. 画面構成

|                |                                 |
|----------------|---------------------------------|
| ステータスバー（常設）    | 接続状態ドット / 状態 / REC / fix / 最終受信 |
| ページ（タブで排他・全画面） | 接続 / 記録 / 解析 / 地図 / 設定          |
| タブバー           |                                 |

旧構成（地図全画面 + 下部ボタンで開くボトムシート）を廃止し、`.page` の排他表示に変更した（`js/tab-ui.js`）。非表示中の `<canvas>` と `Leaflet` はサイズが 0 になるため、**タブ表示時に描き直す**：

| タブ | 表示時の処理                                             |
|----|----------------------------------------------------|
| 記録 | <code>recScatter.redraw()</code>                   |
| 解析 | <code>analysisUI.refresh()</code> （現在のソースで全再描画）    |
| 地図 | <code>mapView.invalidateSize()</code> + 読込データへフィット |

## タブ 表示時の処理

設定 `infoUI.refresh()` (アプリ更新の確認)

`CanvasView.redraw()` を追加し、「サイズ 0 のまま描いた分の描き直し」を共通化した。

## 3. 記録モデル (record → stop → save → load)

record —連続エポック収集→ stop —集計→ [未保存の記録 pending] —save→ IndexedDB

load ← |  
▲  
JSON ファイル —importer.js—

- **stop** は **DB に書かない**。集計結果を pending として返し、記録タブに散布図+集計テキストを表示する。停止理由 (`converged / timeout / maxEpochs / manual`) も pending に持つ。
- **save** で地点名 (既定 `yyyy-mm-dd-xx` の同日連番) とメモを付けて保存し、保存した記録をそのまま「読込データ」に設定する (解析タブ・地図タブへ即反映)。
- 収束自動停止 (`algospec-202607.md` 3.) は維持。自動停止も **stop** と同じ経路で pending になる。
- 記録中は Wake Lock を取得し、画面 OFF / バックグラウンドでは `visibilitychange` で収集を一時停止する。

### サンプルに衛星リストを含める

保存サンプル (`toSample`) に `satellites: [{sys, prn, elev, azim, snr, used}]` を追加した。これにより **保存後・取込後でも任意エポックの SkyPlot / SNR を再現**できる (解析タブのスライダ)。容量は 1 エポックあたり衛星30個程度 = 数KB、120 エポックで数百KB程度に収まる。

## 4. データモデル (IndexedDB: `gnssScopeDB v1`)

DB 名は旧 `gnssDB` から変更した (旧アプリのデータと混ざらないようにするため)。

| ストア                   | キー                                | 内容                                                                                                   |
|-----------------------|-----------------------------------|------------------------------------------------------------------------------------------------------|
| <code>sessions</code> | <code>id</code>                   | <code>{ id, type:'record', label, memo, createdAt, endedAt, summary, (importedAt, sourceId) }</code> |
| <code>points</code>   | <code>id (bySession index)</code> | <code>{ id, sessionId, kind:'record', stats, samples[] }</code>                                      |
| <code>settings</code> | <code>key</code>                  | <code>uere / autoStop / minSec / maxSec / maxEpochs / mapType / trackEnabled / tileCacheMeta</code>  |

- `summary: { lat, lon, altMSL, count, drms, cep50, cep95, stopReason, rxStats }`
- `stats: computeStaticStats()` の戻り値 (中心・中央値・東西/南北標準偏差・DRMS/2DRMS・CEP50/CEP95・標高平均/標準偏差・fix内訳・平均DOP・平均衛星数・平均 **C/N0**・散布図オフセット)
- 設定キーは旧 `staticMinSec` などから `minSec` 等へ改名 (記録が静的測位専用ではなくなったため)。

## エクスポート / インポート

- CSV: 1行1エポック。列に `snr_avg_used_dbhz` (使用衛星の平均 C/N0) を追加。末尾に集計値をコメント行で付ける。
- GPX: 記録の中心を `wpt`、生エポック群を `trk` として出力。
- JSON: `{ app:'gnss-scope', format:1, exportedAt, session, point }`。 `importer.js` が読み戻す。取込時は **必ず新 id** を採番し (同じファイルの二重取込でも上書きしない)、元 id は `sourceId` に残す。 `stats` が欠けている JSON は `computeStaticStats` で再計算する。

## 5. 解析タブ

| 表示                                | ライブ            | 読込データ                        |
|-----------------------------------|----------------|------------------------------|
| SkyPlot / SNR                     | 最新エポック         | スライダで選んだエポック (保存済み衛星リストから再現) |
| DOP (バー+数値・測位モード・衛星数・精度ソース・GST σ) | 最新エポック         | 同上                           |
| DRMS (散布図+集計テキスト)                 | 記録中／直近の記録の暫定集計 | 読込データの確定集計                   |

- ソースはラジオで切替。読込データが無い間は「読込データ」を `disabled` にする。
- 保存サンプル → 表示用エポックへの変換 (`sampleToEpoch`) で `recvAt` を補い、 `canvas-view.js` のキャリーフォワード判定 (`holdDecision`) がライブと同じ経路で働くようにしている。
- 系統別内訳は、ライブでは GSA 由来の `usedBySys` / `inViewBySys` を使い、無い場合 (読込データ) は衛星リストから数える。

## 6. モジュール構成

「1機能=1ファイル」を基準に、**受信 / 解析・記録 / 画面**の3層で分ける (202607 のリファクタリングで 27 ファイル → 17 ファイルに統合。詳細は 9.)。

| 層     | モジュール                        | 責務                                                                              |
|-------|------------------------------|---------------------------------------------------------------------------------|
| エントリ  | <code>app.js</code>          | 受信パイプラインと各タブの結線のみ                                                               |
| 共通    | <code>view-utils.js</code>   | \$ (DOM ショートハンド)・タブ切替・表示フォーマッタ                                                  |
| 受信    | <code>transport.js</code>    | BLE(NUS) 接続・自動再接続 + 開発用モック (同じ <code>onFrame</code> I/F)                        |
| 受信    | <code>nmea.js</code>         | 行復元 ( <code>LineBuffer</code> ) + センテンス解析 + チェックサム ( <code>xorChecksum</code> ) |
| 受信    | <code>epoch.js</code>        | 同一時刻センテンス群 → 1エポック                                                              |
| 受信    | <code>stream-stats.js</code> | 受信品質統計 (\$PPICO 突合を含む)                                                          |
| 解析・記録 | <code>accuracy.js</code>     | 水平精度推定・DRMS/CEP 集計・収束判定                                                         |

| 層     | モジュール                       | 責務                                        |
|-------|-----------------------------|-------------------------------------------|
| 解析・記録 | <code>charts.js</code>      | Canvas 描画（共通土台 + スカイプロット / SNR / 散布図）     |
| 解析・記録 | <code>recorder.js</code>    | record / stop / save の制御                  |
| 解析・記録 | <code>storage.js</code>     | IndexedDB ラッパ                             |
| 解析・記録 | <code>file-io.js</code>     | CSV / GPX / JSON 出力と JSON 取込              |
| 画面    | <code>connect-ui.js</code>  | 接続タブ（受信経路のライフサイクル・受信品質パネル・モック切替）          |
| 画面    | <code>record-ui.js</code>   | 記録タブ（record/stop/save/load・一覧・Wake Lock）  |
| 画面    | <code>analysis-ui.js</code> | 解析タブ（SkyPlot / SNR / DOP / DRMS、ライブ↔読込切替） |
| 画面    | <code>map.js</code>         | 地図タブ（MapView + タブ配線）                      |
| 画面    | <code>settings-ui.js</code> | 設定タブ（各種設定 + アプリ更新確認）                      |
| 画面    | <code>tile-cache.js</code>  | オフラインタイル事前DL（TileCache + 設定タブ内のUI）        |

- `app.js` は結線のみ。DOM 操作はタブごとの `*-ui.js` が持つ。
- 読込データ（loaded）は `app.js` が保持し、`setLoaded()` で解析タブ・地図タブへ配る。
- 廃止：旧「ライブ」タブ（記録タブの「現在の測位値」へ統合）／旧「情報」タブ（設定タブへ統合）／スナップショット記録。

## 7. PWA・オフラインタイル

- シェルキャッシュ名は `gnss-scope-shell-v2`（版数を兼ねる）。`settings-ui.js` がこの名前を 正規表現で読み取ってサーバー上の最新版と比較する。`APP_SHELL` には全モジュールを列挙する。
- 地理院タイルは `cache-first` でキャッシュ `gsi-tiles` に格納する。Service Worker（実行時の 取得）と事前ダウンロード（`tile-cache.js`）が**同じキャッシュ名を共有**する点が必要。
- 未キャッシュのタイルをオフラインで要求した場合は 504 を返し、Leaflet 側でグレー表示にする。

### タイルマニフェスト（`data/tile_manifest.json`）

事前ダウンロードの対象を座標リストで持つ（Format A）。マニフェストは `data/` を唯一の置き場とし、ルート直下に複製を置かない。

```
{
  "version": "2026-06-01",
  "layers": {
    "<任意のキー>": { "z": 17, "tiles": [[x, y], [x, y], ...] }
  }
}
```

- `z` はズームレベル、`tiles` は当該ズームのタイル座標 `[x, y]` の配列。

- URL は選択中の地図種別のテンプレート (`GSI_LAYERS[mapType].url`) の `{z}/{x}/{y}` を置換して生成する。したがって地図種別を変えると、その種別のタイルは再取得が必要。
- `version` は取得時に `settings.tileCacheMeta` へ記録し、オフラインでも取得済み内容を表示できるようにする。

## 8. リファクタリング (202607・機能変更なし)

ファイル数が多く (27)、似た処理が分散していたため、機能を変えずに統合・整理した。

### 8-1. ファイルの統合 (27 → 17)

| 統合後                         | 統合したファイル                                                                                              | まとめた理由                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <code>nmea.js</code>        | <code>nmea-parser</code> + <code>line-buffer</code>                                                   | どちらも「生バイト列 → 1センテンス」の下位層。チェックサム計算を共用できる             |
| <code>transport.js</code>   | <code>ble-client</code> + <code>mock-feeder</code>                                                    | 同じ <code>onFrame</code> I/F で差し替える受信経路。実機／モックは対の関係  |
| <code>epoch.js</code>       | <code>epoch-assembler</code> (改名)                                                                     | 名前をファイルの役割に合わせた                                     |
| <code>charts.js</code>      | <code>canvas-view</code> + <code>sky-plot</code> + <code>snr-chart</code> + <code>scatter-plot</code> | すべて Canvas 描画。共通土台と3ビューを同居させ、共通ヘルパを内部化              |
| <code>file-io.js</code>     | <code>exporter</code> + <code>importer</code>                                                         | 記録のファイル入出力。JSON 形式の定義を1か所に持てる                       |
| <code>view-utils.js</code>  | <code>format</code> + <code>tab-ui</code> + <code>\$</code> ヘルパ                                       | 画面共通のユーティリティ。 <code>\$</code> を各モジュールへ引数で配る配線が不要になる |
| <code>map.js</code>         | <code>map</code> + <code>map-ui</code>                                                                | 地図本体とタブ配線は常に一緒に変更する                                 |
| <code>tile-cache.js</code>  | <code>tile-cache</code> + <code>tile-ui</code>                                                        | タイル取得と、その進捗UIは一体                                    |
| <code>settings-ui.js</code> | <code>settings-ui</code> + <code>info-ui</code>                                                       | アプリ更新確認は設定タブの中にある機能                                 |
| <code>record-ui.js</code>   | <code>record-ui</code> + <code>wake-lock</code>                                                       | 「記録中の画面維持」は記録機能の一部                                  |
| <code>connect-ui.js</code>  | ( <code>app.js</code> から分離・新規)                                                                        | 接続制御と受信品質パネルを <code>app.js</code> から切り出し            |

`app.js` は結線のみに絞り 330 行 → 162 行。各 UI モジュールは自分が使う描画ビューを 自前で生成するようして、`app.js` からの受け渡しを減らした。

### 8-2. 重複処理の共通関数化

| 対象               | 変更                                                                                   |
|------------------|--------------------------------------------------------------------------------------|
| チェックサム XOR 計算    | パーサの検証とモックの文生成で重複 → <code>xorChecksum()</code> に集約                                   |
| HTML / XML エスケープ | <code>escapeHtml</code> と <code>escXml</code> の2実装 → <code>escapeMarkup()</code> に統合 |
| キャリーフォワード判定      | スカイプロットと SNR で同じ 12 行 → <code>CanvasView._holdOrClear()</code>                       |

| 対象             | 変更                                                                                                                        |
|----------------|---------------------------------------------------------------------------------------------------------------------------|
| 十字・リング・方位ラベル描画 | スカイプロットと散布図で別実装 → <code>drawCross</code> / <code>strokeCircle</code> / <code>fillCircle</code> / <code>drawCompass</code> |
| タイル URL 列挙     | 件数カウントとダウンロードで二重ループ → <code>TileCache.tileUrls()</code>                                                                   |
| 設定行の保存処理       | 6 個のほぼ同型リスナ → 定義テーブル + 2 ループ                                                                                              |
| 「使用/視野内」表示     | 記録タブと解析タブで重複 → <code>satsText()</code>                                                                                    |
| 追従トグルの通知       | 3 か所で個別に UI 反映 → <code>MapView.setFollow()</code> に集約                                                                     |
| 一覧の行アクション      | if 連結 → <code>runAction()</code> + エクスポートの対応表                                                                             |

### 8-3. 未使用コードの削除

- エポックが持っていて誰も読まない値：`geoidSep` / `status` (RMC) / `altStd` (GST) / `sentenceTypes` / `invalidCount` (不正文の件数は `stream-stats.js` が数えている)。パーサ側の対応フィールドも併せて削除。
- 使われていない設定可能性：`NmeaBle` の `deviceName` / `maxBackoffMs` (モジュール定数へ)、`MockFeeder` の `hz`、`TileCache` の `zoomFilter` と `countTiles()`。
- 読まれないフィールド・引数：`NmeaBle.status` と `_setStatus()`、`WakeLockManager` の `isSupported()` と `onChange` の第1引数、SNR チャートの到達不能ガード。
- 外へ出す必要のない export の内部化：`HOLD_MS` / `CanvasView` / `stopReasonText` / `STOP_REASON` / `rxStatsText` / `JSON_FORMAT`。

なお `computeStaticStats` の `avgPdop` / `avgVdop` は画面に出ないが、**JSON エクスポート**で利用者へ渡る値なので残している。

## 9. テスト

`node tools/check-syntax.mjs` に以下を追加した。

- 使用衛星の平均 C/N0 (未使用衛星を除外すること)
- `formatStats` が未保存の記録でも生成できること
- 記録フロー：`stop` で集計しても DB に書かないこと／サンプルに衛星リストが残ること／`save` で `sessions`・`points` に保存され `summary` に集計値が入ること